DevOps pipeline for a course review web app

DD2482 project report

Georgi Tsonchev Hristov (georgith@kth.se), Cesar Aceves Hernández (cesarah@kth.se)

1. Introduction

Our project is a small web app where students can leave anonymous reviews of KTH courses. It is a Flask app with a SQLite database and one page with a form and the list of reviews. We kept the app simple on purpose, since the project is about the pipeline around it. Everything is self-hostable, with Docker and Terraform anyone can run the same setup on their own machine.

The code is in a public GitHub repository: https://github.com/tctev/course-reviews. The link to the deployed app is in our submission PR.

2. Architecture

The app runs with gunicorn inside a Docker container on port 8000. The image is based on `python:3.13-slim` and does not run as root. The database is the file `/data/reviews.db`, and `/data` is a Docker volume, so when we deploy a new version the container is replaced but the reviews stay.

The system has these parts:

- GitHub, for the code, pull requests, GitHub Actions and the container registry (GHCR).
- A CI workflow that runs on every pull request on GitHub-hosted runners.
- A CD workflow that runs on every push to `main`. The image is built on a GitHub-hosted runner and the deploy runs on our self-hosted runner.
- Terraform with the `kreuzwerker/docker` provider, which describes the network, the volume, the image and the container.
- An Azure VM (Ubuntu 24.04, Azure for Students) where the self-hosted runner and the app run.

A change goes like this: we open a PR and CI runs. The PR can only be merged when CI passes. After the merge the image is built, tagged with the commit SHA and pushed to GHCR. Then the self-hosted runner runs `terraform apply` with this tag and checks that the app answers.

3. Continuous integration

The CI workflow runs on `pull_request`. It installs the dev dependencies and runs:

1. `ruff check` and `ruff format --check` for linting and formatting.
2. `pytest`. The tests post a review, check that bad input is rejected (wrong course code, rating outside 1-5, empty or too long text) and that HTML is escaped.
3. `terraform fmt -check`, `terraform init -backend=false` and `terraform validate`.
4. `docker build`, so a broken Dockerfile shows up before merging.

`main` is protected. Every change needs a PR and the `test` job has to pass, also for admins, so we can't push to `main` directly. The workflow token only has `contents: read`. We added this after CodeQL warned that the workflow didn't limit the permissions of the `GITHUB_TOKEN`.

4. Continuous deployment

The CD workflow runs on push to `main` and has two jobs.

The `build` job runs on `ubuntu-latest`. It logs in to GHCR with the `GITHUB_TOKEN` and pushes `ghcr.io/tctev/course-reviews:<commit sha>`. We tag with the SHA instead of `latest`, so

every deploy uses one exact image and the tag shows which commit is running. To go back to an older version we can run the apply again with an older SHA.

The `deploy` job needs `build` and runs on the self-hosted runner. It logs in to GHCR, installs Terraform, pulls the image and runs `terraform apply -var image=...`. At the end a smoke test with `curl` against `localhost:8000` makes the job fail if the container doesn't answer. The curl call retries for a few seconds, because gunicorn is sometimes not ready right after the container starts. The workflow uses `concurrency: deploy`, so two deploys never run at the same time on the VM.

The Terraform state is a local file in the home folder of the runner user. It can't be in the workspace, because `actions/checkout` cleans the workspace on every run and then Terraform would forget the container and try to create it again.

5. Infrastructure as Code

`terraform/main.tf` has four resources: a `docker_network`, a `docker_volume` for the database, a `docker_image` and a `docker_container`. The image is a variable. In CD it is the GHCR image with the SHA, and the default is `course-reviews:local`. So anyone with Docker can run `docker build -t course-reviews:local .` and `terraform apply` and get the same setup on their own computer, which was one of the goals in our proposal.

When the image tag changes, Terraform only replaces the container and keeps the network and the volume, so the database is not lost. The provider version is pinned in `.terraform.lock.hcl`.

6. Quality and security automation

- CodeQL runs on PRs, on pushes to `main` and once a week, on the Python code and on the workflow files. It found the missing token permissions mentioned above, and now there are no open alerts.
- Dependabot checks pip, GitHub Actions, the Docker base image and the Terraform provider every week.
- Ruff and pytest run in CI (section 3).

7. Design decisions

We used GitHub Actions because our code was already on GitHub. CI, CD and the registry are in the same place and we don't have to run our own CI server.

For Infrastructure as Code we picked Terraform with the Docker provider. Kubernetes or a full cloud setup would be too much for one container, and with the Docker provider the same file works on a laptop and on the server.

The deploy needs a self-hosted runner, because Terraform has to talk to the Docker daemon on the machine where the app runs and a GitHub-hosted runner is deleted after the job. At first the runner was on Georgi's laptop. It worked, but we had to install Docker, Terraform and the runner on a personal computer, and deploys only ran when the laptop was on. So we moved the runner to an Azure VM, where we set up these dependencies once on a clean machine. The VM is just one host, the setup itself doesn't depend on Azure. The student subscription only allows a few regions, so the VM is in Denmark East. It has 1 GB of RAM, which is enough because the image is built on GitHub and the VM only pulls it, but we added 2 GB of swap to be safe.

We use SQLite in a volume because it is one file and needs no separate database server. For a course review app with little traffic that is enough.

8. Limitations and trade-offs

- Everything runs on one VM. If it is down, the image is still built and pushed but the deploy waits until the runner is back. There is no failover and no staging environment.

- The Terraform state is a local file without locking or backup, in a real project it should be in a remote backend. We had this problem once: an old deploy left a Docker network without state, the next `terraform apply` failed because the name was already used, and we had to remove it by hand.
- SQLite doesn’t work with several replicas of the app, so we can’t scale out.
- The app uses plain HTTP on port 8000, without HTTPS or a reverse proxy.
- The runner user is in the `docker` group, which is almost the same as root on the VM. Since the repository is public, only deploy jobs from `main` run there and pull requests (also from forks) run on GitHub-hosted runners. Still, anyone who can merge to `main` can run code on the VM.
- The tests are small. They cover the form and the input validation, and the deploy is only checked by the smoke test.